

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..16

CUSTOMIZATIONS

 Copy as Markdown

## Customize Facebook sign-in

This page covers additional configuration options for the Facebook identity provider beyond the basic setup. It also covers how to customize the sign-in UI. You can use the `FacebookSignInWidget` to display the Facebook sign-in flow in your own custom UI, or the `FacebookAuthController` to build a completely custom authentication interface.

### Server configuration

Common configuration options for the Facebook provider. For more details on all options, check the `FacebookIdpConfig` in-code documentation.

### Load the credentials yourself

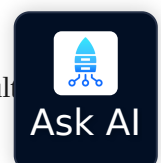
The setup guide uses `FacebookIdpConfigFromPasswords`, which reads `facebookAppId` and `facebookAppSecret` from your password store. To control the loading yourself, use `FacebookIdpConfig` and pass the values directly:

```
FacebookIdpConfig(  
  appId: myAppId,  
  appSecret: myAppSecret,  
)
```

Both classes accept the same optional callbacks, such as `facebookAccountDetailsValidation` and `getExtraFacebookInfoCallback`, shown below. The examples on this page use `FacebookIdpConfigFromPasswords`, the class from the setup guide.

### Custom account validation

You can customize the validation for Facebook account details before allowing sign-in. By default, validation checks that the received account details contain a non-empty `userIdentifier`.



The default validation logic:

```
static void validateFacebookAccountDetails(  
    final FacebookAccountDetails accountDetails,  
) {  
    if (accountDetails.userIdentifier.isEmpty) {  
        throw FacebookUserInfoMissingDataException();  
    }  
}
```

To customize validation, provide your own `facebookAccountDetailsValidation` function:

```
final facebookIdpConfig = FacebookIdpConfigFromPasswords(  
    // Optional: Custom validation for Facebook account details  
    facebookAccountDetailsValidation: (FacebookAccountDetails accountDetails) {  
        // Throw an exception if account doesn't meet custom requirements  
        if (accountDetails.userIdentifier.isEmpty) {  
            throw FacebookUserInfoMissingDataException();  
        }  
        // Example: Require email to be present  
        if (accountDetails.email == null || accountDetails.email!.isEmpty) {  
            throw FacebookUserInfoMissingDataException();  
        }  
    },  
);
```

#### NOTE

Users may choose not to share their email or other information during the Facebook login flow. Adjust your validation function carefully to avoid blocking legitimate users.

## FacebookAccountDetails

The `facebookAccountDetailsValidation` callback receives a `FacebookAccountDetails` record with the following properties:

| Property                    | Type                 | Description                                 |
|-----------------------------|----------------------|---------------------------------------------|
| <code>userIdentifier</code> | <code>String</code>  | The Facebook user's unique identifier (UID) |
| <code>email</code>          | <code>String?</code> | The user's email address (may be null)      |
| <code>fullName</code>       | <code>String?</code> | The user's full name from Facebook          |

| Property               | Type                 | Description                     |
|------------------------|----------------------|---------------------------------|
| <code>firstName</code> | <code>String?</code> | The user's first name           |
| <code>lastName</code>  | <code>String?</code> | The user's last name            |
| <code>image</code>     | <code>Uri?</code>    | URL to the user's profile image |

Example of accessing these properties:

```
facebookAccountDetailsValidation: (accountDetails) {
  print('Facebook UID: ${accountDetails.userIdentifier}');
  print('Email: ${accountDetails.email}');
  print('Display name: ${accountDetails.fullName}');
  print('Profile image: ${accountDetails.image}');

  // Custom validation logic
  if (accountDetails.email == null) {
    // Handle case where user didn't share email
  }
},
```

#### INFO

The properties available depend on the permissions requested and what the user consented to share.

## Accessing Facebook APIs on the server

### CAUTION

The `getExtraFacebookInfoCallback` below runs on **every** sign-in, not only the first. Cache what you fetch, and guard external calls with `try/catch` so a provider outage does not block sign-in.

On the server side, you can access Facebook APIs using the access token. The

`getExtraFacebookInfoCallback` in `FacebookIdpConfig` receives the access token and can be used to call Facebook Graph APIs:

```
import 'package:http/http.dart' as http;

final facebookIdpConfig = FacebookIdpConfigFromPasswords(
  // Optional: Extract additional info from Facebook Graph APIs
  getExtraFacebookInfoCallback: (session, {
    required accountDetails,
    required accessToken,
    required transaction,
```

```

}) async {
  // Use accessToken to call Facebook Graph APIs and store additional info
  // Example: Access user's friends list
  final response = await http.get(
    Uri.https('graph.facebook.com', '/v21.0/me/friends'),
    headers: {'Authorization': 'Bearer $accessToken'},
  );
  // Process response and store additional info in the database
},
);

```

## Reacting to Facebook account creation

You can use the `onAfterFacebookAccountCreated` callback to run logic after a new Facebook account has been created and linked to an auth user. This callback is only invoked for new accounts, not for returning users.

This callback is complementary to the core `onAfterAuthUserCreated` callback to perform side-effects that are specific to a login on this provider, like storing analytics, sending a welcome email, or storing additional data.

```

final facebookIdpConfig = FacebookIdpConfigFromPasswords(
  onAfterFacebookAccountCreated: (
    session,
    authUser,
    facebookAccount, {
      required transaction,
    }) async {
    // e.g. store additional data, send a welcome email, or log for analytics
  },
);

```

### INFO

This callback runs inside the same database transaction as the account creation. Throwing an exception inside this callback will abort the process. If you perform external side-effects, make sure to safeguard them with a try/catch to prevent unwanted failures.

### CAUTION

Scopes you assign here with `AuthServices.instance.authUsers.update()` do not apply to the login that is already in progress, because token issuance uses the scopes loaded before this callback runs. They take effect the next time the user signs in. To assign scopes at creation time instead, use `onBeforeAuthUserCreated` together with `getExtraFacebookInfoCallback`, which runs before the auth user is created.

## FacebookIdpConfig parameter reference

| Parameter                                     | Type                                              | Required | Description                                                                                                                                                                                            |
|-----------------------------------------------|---------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>appId</code>                            | <code>String</code>                               | Yes      | The app ID from your Facebook Developer app.                                                                                                                                                           |
| <code>appSecret</code>                        | <code>String</code>                               | Yes      | The app secret from your Facebook Developer app.                                                                                                                                                       |
| <code>facebookAccountDetailsValidation</code> | <code>FacebookAccountDetailsValidation</code>     | No       | Custom validation callback for Facebook account details before allowing sign-in. Throws an exception to reject the account. Defaults to validating only that <code>userIdentifier</code> is non-empty. |
| <code>getExtraFacebookInfoCallback</code>     | <code>GetExtraFacebookInfoCallback?</code>        | No       | Callback that receives the access token after sign-in, allowing you to call the Facebook Graph API and store extra user data. Runs on every sign-in.                                                   |
| <code>onAfterFacebookAccountCreated</code>    | <code>AfterFacebookAccountCreatedFunction?</code> | No       | Callback invoked after a new Facebook account is created and linked to an auth user. Fires only for new accounts.                                                                                      |

## App configuration

These options are configured in your Flutter app rather than on the server.

### Configuring Facebook sign-in on the app

When using the external `serverpod_auth_idp_flutter_facebook` package, you can configure the App ID in your Flutter application.

You can pass the App ID directly when initializing the Facebook sign-in service:

```
await client.auth.initializeFacebookSignIn(  
  appId: 'YOUR_FACEBOOK_APP_ID',  
);
```

If the `appId` value is not supplied when initializing the service, the provider will automatically fetch it from the `FACEBOOK_APP_ID` environment variable. You can set this variable at build time using the `--dart-define` option:

```
flutter run -d <device> \
  --dart-define="FACEBOOK_APP_ID=your_app_id"
```

This approach is useful when you need to:

- Set the App ID for web and macOS builds, where it is read from Dart. On Android and iOS the Facebook SDK reads it from the native configuration files instead.
- Avoid committing App IDs to version control.
- Configure different credentials for different build environments (development, staging, production).

#### TIP

You can also set these environment variables in your IDE's run configuration or CI/CD pipeline to avoid passing them manually each time.

## Accessing Facebook APIs

The default setup allows access to basic user information, such as `name` and `email`. You may require additional permissions to access other Facebook APIs, such as accessing a user's friends, posts, or pages.

The default permissions requested are:

- `email`: Access to user's email address.
- `public_profile`: Access to user's basic profile information.

To request additional permissions, you will need to:

- Ensure the required permissions are configured in your Facebook App settings (navigate to **Use cases > Customize > Permissions and features** in the [Facebook App Dashboard](#)).
- Request access to the permissions when signing in. Do this by setting the `permissions` parameter of the `FacebookSignInWidget` or `FacebookAuthController`.

A full list of available permissions can be found in the [Facebook permissions reference](#).

#### INFO

Adding additional permissions may require App Review depending on the sensitivity of the requested permissions and your app's use case.

## Customize the sign-in button

If you render the button inside `SignInWidget`, see [Styling the buttons](#) for how its `buttonStyle` overrides the parameters set here.

#### INFO

The `SignInWidget` automatically detects and displays the Facebook sign-in flow when the `serverpod_auth_idp_flutter_facebook` package is installed and initialized. The Facebook provider registers itself dynamically with the sign-in widget.

## Using the `FacebookSignInWidget`

The `FacebookSignInWidget` handles the complete Facebook sign-in flow for iOS, Android, web, and macOS.

You can customize the widget's appearance and behavior:

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import
'package:serverpod_auth_idp_flutter_facebook/serverpod_auth_idp_flutter_facebook.dart';

FacebookSignInWidget(
  client: client,
  // Button customization. The values shown are the defaults.
  style: FacebookButtonStyle.blue, // or white
  size: SignInButtonSize.large, // or medium, small
  text: SignInButtonTextVariant.continueWith, // or signInWith, signUpWith, signIn
  shape: SignInButtonShape.pill, // or rounded, rectangular
  logoAlignment: SignInButtonLogoAlignment.center, // or left
  minimumWidth: 240, // at most 400
  textStyle: null, // TextStyle for the label

  // Permissions to request from Facebook
  // These are the default permissions.
  permissions: const ['email', 'public_profile'],

  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
)
```

## Build a custom UI with `FacebookAuthController`

For more control over the UI, you can use the `FacebookAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import
'package:serverpod_auth_idp_flutter_facebook/serverpod_auth_idp_flutter_facebook.dart';

final controller = FacebookAuthController(
  client: client,
  onAuthenticated: () {
    // Do something when the user is authenticated.
    //
    // NOTE: You should not navigate to the home screen here, otherwise
    // the user will have to sign in again every time they open the app.
  },
  onError: (error) {
    // Handle errors
  },
  permissions: const ['email', 'public_profile'],
);

// Initiate sign-in
await controller.signIn();
```

## FacebookAuthController state management

Your widget should render the appropriate UI based on the `state` property of the controller. You can also use the below state properties to build your UI:

```
// Check current state
final state = controller.state; // FacebookAuthState enum

// Check if loading
final isLoading = controller.isLoading;

// Check if authenticated
final isAuthenticated = controller.isAuthenticated;

// Get error message
final errorMessage = controller.errorMessage;

// Get error object
final error = controller.error;

// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

## FacebookAuthController states

- `FacebookAuthState.initializing` - Controller is initializing.
- `FacebookAuthState.idle` - Ready for user interaction.
- `FacebookAuthState.loading` - Processing a sign-in request.
- `FacebookAuthState.error` - An error occurred.
- `FacebookAuthState.authenticated` - Authentication was successful.

## Related

- [Setup](#): set up the Facebook identity provider on the server and in your app.
- [Troubleshooting](#): fix common Facebook sign-in errors.
- [UI components](#): style the sign-in widget and its provider buttons.
- [Working with users](#): manage auth users and react to account events.